

Universidad CENFOTEC
Escuela de Software

Primer estudio de caso

Aplicación de las pilas al análisis de cadenas de impresión

Estructuras de Datos
Docente facilitador: Romario Salas Cerdas

Estudiante: Braden Lee Knuter Cordoba

Fecha de entrega: 11 de octubre del 2026

Repositorio del proyecto en GitHub:
<https://github.com/B0W3YYY/CasoDeEstudio1-EstructuraDeDatos>

1. Alternativa seleccionada

De las dos aplicaciones propuestas en la consigna, este trabajo desarrolla el análisis de cadenas de impresión. El programa recibe una expresión como las que se escriben dentro de una instrucción de impresión y determina si está bien formada. Los tokens que contempla son los tres que pide el enunciado: cadenas literales delimitadas por comillas dobles, variables de cadena identificadas por su nombre y el operador de concatenación +. A ellos se agregan los paréntesis de agrupación. Cada uno de estos tokens es el dato que guarda un nodo de la pila, como establece la consigna para esta alternativa.

Una expresión válida es "Hola, " + nombre + ("!" + salto). Una inválida es ("Total: " + monto, porque el paréntesis que se abrió nunca se cerró. El problema consiste en decidir si la secuencia de tokens forma una expresión completa: que toda cadena literal termine con su comilla, que cada paréntesis de apertura encuentre su cierre y que los operandos y el operador se alternen como corresponde. La pila se programó en Java con nodos enlazados, sin usar las clases de colección del lenguaje.

2. Por qué una pila es la estructura de datos adecuada

El requerimiento central del análisis de cadenas de impresión es reconocer la estructura de la expresión, y esa estructura está anidada. Cuando aparece un paréntesis de cierre, la única parte de la expresión con la que puede combinarse es la más reciente que siga pendiente. En la expresión ("a" + (b + "c")), el primer paréntesis de cierre no corresponde al paréntesis inicial sino al segundo, porque es el último que quedó abierto. Ese comportamiento, donde el último elemento que entra es el primero que sale, es la política LIFO (*Last In, First Out*) que define a una pila (Goodrich, Tamassia y Goldwasser, 2014).

La correspondencia entre el problema y la estructura es formal. El lenguaje de los paréntesis balanceados no es un lenguaje regular, de modo que no puede reconocerse con un autómata de estados finitos: la profundidad de anidamiento es ilimitada y un autómata finito no puede llevar una cuenta arbitraria. Perteneció a los lenguajes independientes del contexto, y el modelo de máquina que reconoce esa clase es el autómata de pila. Los compiladores resuelven este problema con el análisis sintáctico ascendente por desplazamiento y reducción, que trabaja sobre una

pila de tokens (Aho, Lam, Sethi y Ullman, 2007).

El programa aplica esa misma técnica. Cada token se inserta en la pila y espera allí hasta que puede combinarse con sus vecinos. Cuando la cima forma el patrón operando, +, operando, o cuando llega un paréntesis de cierre sobre un paréntesis de apertura y un operando, esos nodos se retiran y se reemplazan por un único nodo EXPR que representa la parte ya reconocida. Como las reducciones siempre ocurren en la cima, los tokens más recientes se combinan primero, que es lo que exige el anidamiento. La expresión es válida si al terminar queda un solo operando en la pila.

Las alternativas quedan descartadas por razones concretas. Un contador de paréntesis abiertos detectaría que faltan cierres, pero no conservaría los tokens ni podría informar en qué posición quedó abierto el paréntesis pendiente. Una lista o un arreglo recorridos libremente resolverían el problema, aunque ofrecerían accesos que el algoritmo no necesita y que, usados por error, combinarían tokens que no corresponden. Una cola, con su política FIFO, combinaría primero los tokens más antiguos, que es lo contrario de lo que pide el anidamiento. La misma idea aparece en el algoritmo de dos pilas de Dijkstra para evaluar expresiones aritméticas con paréntesis (Sedgewick y Wayne, 2011). La pila expone solo las operaciones que el problema requiere, y esa restricción es parte de lo que vuelve correcto al algoritmo.

3. Orden, seguridad y eficiencia

3.1 Orden

La pila impone el orden de resolución sin que el programador tenga que administrarlo. Al recorrer la expresión de izquierda a derecha, cada token se apila y las reducciones se hacen siempre en la cima, por lo que las partes internas de la expresión se resuelven antes que las externas. El estado de la pila describe en todo momento lo que falta por completar: cada paréntesis de apertura que permanece en ella es un contexto todavía abierto, y un + en la cima indica que se espera un operando. Esa información permite emitir un diagnóstico preciso en lugar de un aviso genérico de error.

3.2 Seguridad

La seguridad que aporta la pila opera en dos niveles. En el del algoritmo, la interfaz reducida de la estructura vuelve imposibles ciertos errores: como no existe un método

para leer o modificar un elemento intermedio, una reducción solo puede combinar los tokens de la cima y no hay forma de emparejar un cierre con un paréntesis que no sea el correcto. En el del programa, la pila de este proyecto está diseñada para no interrumpir la ejecución ante un uso indebido. Los métodos `pop()` y `peek()` devuelven `null` cuando la pila está vacía en lugar de lanzar una excepción, de modo que quien los invoca comprueba el resultado y muestra un mensaje al usuario. Por eso una entrada mal formada como "Hola" + nombre) produce una explicación y no una terminación abrupta.

El análisis de la expresión es además una forma de validación de entrada. Revisar la estructura de una cadena antes de procesarla es una práctica de seguridad conocida, porque una cadena con comillas sin cerrar o con paréntesis desbalanceados puede alterar el significado del texto que la sigue, problema emparentado con las inyecciones de código que aparecen al construir instrucciones concatenando texto sin verificarlo.

3.3 Eficiencia

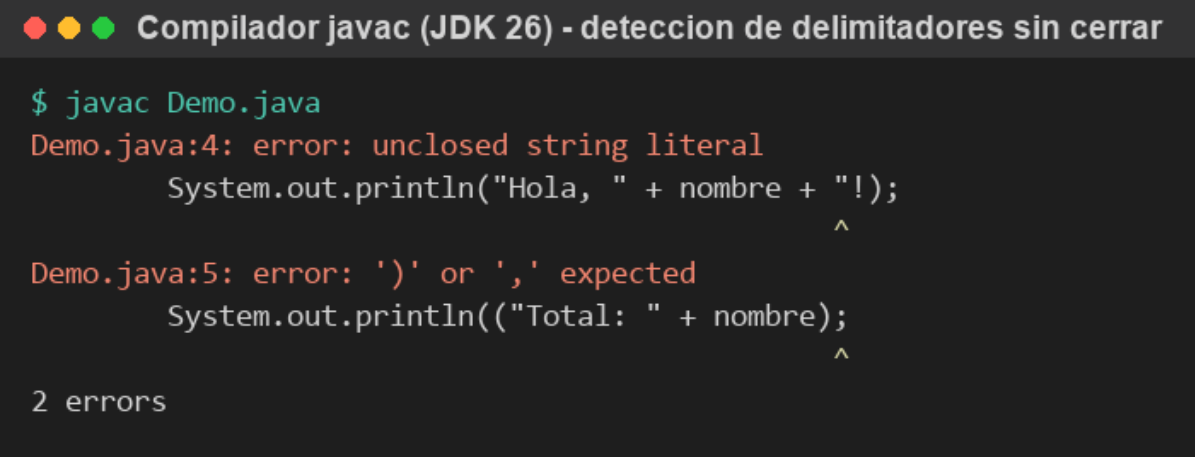
Las operaciones de la pila trabajan únicamente sobre la cima y no recorren la estructura, así que su costo es constante, $O(1)$ (Cormen, Leiserson, Rivest y Stein, 2022). Cada token se inserta una vez y se retira a lo sumo una vez, y cada reducción reemplaza varios nodos por uno solo, de manera que el análisis completo de una expresión de longitud n se resuelve en tiempo $O(n)$, en una sola pasada. La memoria tampoco depende de la longitud de la expresión: como las concatenaciones se reducen apenas se completan, en cada nivel de anidamiento quedan como máximo un paréntesis, un operando y un operador pendientes, y el tamaño de la pila es proporcional a la profundidad de anidamiento, que en una cadena de impresión real es muy pequeña. Al estar construida con nodos enlazados, la pila crece y decrece sin reservar espacio por anticipado ni copiar elementos.

4. Ejemplo concreto en software de sistema

El caso más directo de esta funcionalidad en software de sistema es el compilador de Java, `javac`, incluido en el JDK. Antes de traducir un programa, el compilador divide el texto fuente en tokens y verifica que formen instrucciones correctas. Es la misma división en dos etapas que hace el analizador de este proyecto: una cadena literal sin cerrar se detecta al leer los tokens, y los paréntesis desbalanceados se detectan en

el análisis sintáctico, que se apoya en un autómata de pila (Aho et al., 2007). La especificación del lenguaje establece en su sección 3.10.5 que un literal de cadena está delimitado por comillas dobles y que constituye un error de compilación que aparezca un terminador de línea entre la comilla de apertura y la de cierre, lo que vuelve detectable una comilla sin cerrar (Oracle, 2023).

Para ilustrar el comportamiento se compiló un archivo que contiene los dos errores de forma deliberada. La figura 1 muestra la salida de javac:



```

●●● Compilador javac (JDK 26) - deteccion de delimitadores sin cerrar

$ javac Demo.java
Demo.java:4: error: unclosed string literal
    System.out.println("Hola, " + nombre + "!);
                                   ^
Demo.java:5: error: ')' or ',' expected
    System.out.println(("Total: " + nombre);
                                   ^

2 errors
```

Figura 1. Salida del compilador javac del JDK 26 al procesar un archivo con una cadena literal sin cerrar y un paréntesis sin cerrar. El compilador señala la línea y marca con el símbolo ^ la posición exacta del problema.

El compilador no se limita a indicar que el archivo es incorrecto. Identifica el tipo de error (*unclosed string literal*, *)' expected*) y la posición donde ocurre. Esa precisión es posible porque conserva los contextos abiertos, de modo que al terminar de leer sabe cuáles quedaron pendientes. El programa desarrollado en este estudio de caso reproduce el mismo comportamiento sobre las dos expresiones equivalentes, como se observa en la figura 2, donde además se muestra cada push y cada pop junto con el contenido de la pila de tokens después de cada paso.

```
AnalizadorCadenas.java - mismo diagnóstico con la pila de tokens

$ java -cp out Main
Analizando: "Hola, " + nombre + "!"
  push "Hola, " (cadena literal, posicion 0)
    -> [cima] "Hola, " [base]
  push + (operador de concatenacion, posicion 9)
    -> [cima] + | "Hola, " [base]
  push nombre (variable de cadena, posicion 11)
    -> [cima] nombre | + | "Hola, " [base]
  pop de operando, + y operando, push EXPR
    -> [cima] EXPR [base]
  push + (operador de concatenacion, posicion 18)
    -> [cima] + | EXPR [base]
  ERROR en la posicion 20: la comilla de apertura no fue cerrada.
Analizando: ("Total: " + nombre
  push ( (parentesis de apertura, posicion 0)
    -> [cima] ( [base]
  push "Total: " (cadena literal, posicion 1)
    -> [cima] "Total: " | ( [base]
  push + (operador de concatenacion, posicion 11)
    -> [cima] + | "Total: " | ( [base]
  push nombre (variable de cadena, posicion 13)
    -> [cima] nombre | + | "Total: " | ( [base]
  pop de operando, + y operando, push EXPR
    -> [cima] EXPR | ( [base]
  ERROR en la posicion 0: el parentesis de apertura no fue cerrado.
Programa finalizado. Hasta pronto.
```

Figura 2. Salida del analizador desarrollado (opción 2 del menú) sobre los mismos dos casos defectuosos. La traza muestra cada token que entra en la pila, cada reducción a EXPR y el diagnóstico final con la posición del error.

Al comparar ambas figuras se observa que la lógica es la misma: la comilla sin cerrar se detecta al leer el token, y el paréntesis sin cerrar se detecta porque al terminar el recorrido sigue en la pila. El mismo mecanismo aparece en los editores de código, que resaltan el paréntesis correspondiente al que tiene el cursor, y en los intérpretes interactivos, que mantienen abierta la entrada mientras quede un paréntesis sin cerrar.

5. Representación gráfica de las operaciones

Las figuras siguientes representan la pila como una columna de casillas. Cada casilla es un nodo de la clase Pila que guarda un token, y la casilla superior ocupada es la cima. Las casillas verdes son nodos EXPR producidos por una reducción. La figura 3 muestra las operaciones fundamentales sobre la pila de tokens.

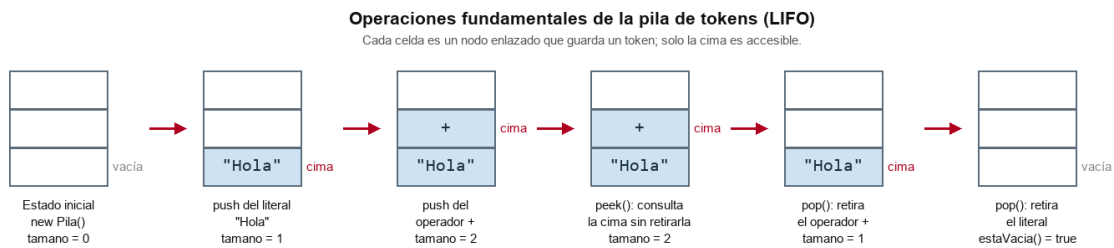


Figura 3. Operaciones fundamentales de la pila de tokens: estado inicial, dos operaciones push, una consulta peek que no modifica la estructura y dos operaciones pop que la devuelven al estado vacío.

La figura 4 presenta la traza completa del análisis de una expresión válida. Los pasos coinciden con la salida que produce la opción 2 del menú.

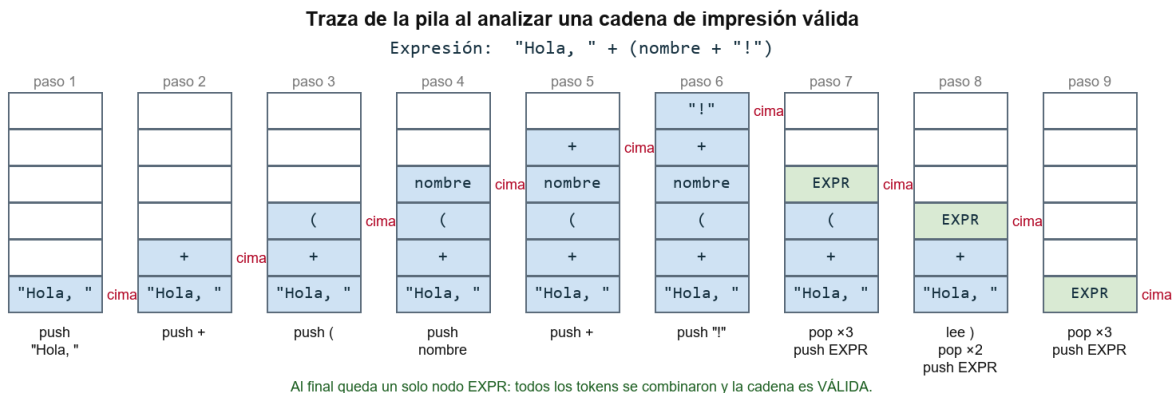
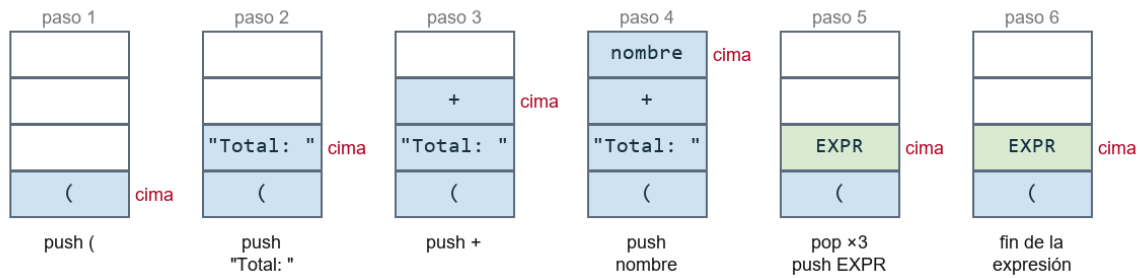


Figura 4. Traza del análisis de la expresión "Hola, " + (nombre + "!"). Las reducciones de los pasos 7, 8 y 9 dejan un solo nodo EXPR, lo que confirma que la cadena es válida.

La figura 5 corresponde al caso inválido. Al terminar el recorrido quedan dos nodos en la pila, y el paréntesis que sigue en ella indica cuál quedó abierto y en qué posición.

Detección de un paréntesis sin cerrar

Expresión: ("Total: " + nombre



Al terminar quedan dos nodos: el (de la posición 0 nunca se cerró y la cadena es INVÁLIDA.

Figura 5. Análisis de la expresión inválida ("Total: " + nombre. Al finalizar, el paréntesis de la posición 0 continúa en la pila, por lo que la cadena se rechaza.

La figura 6 resume el flujo de control del método que realiza el análisis e incluye las operaciones que la funcionalidad necesita además de las fundamentales: la lectura y clasificación de cada token, la reducción de los patrones completos y la verificación final.

Diagrama de flujo del método analizar() de AnalizadorCadenas

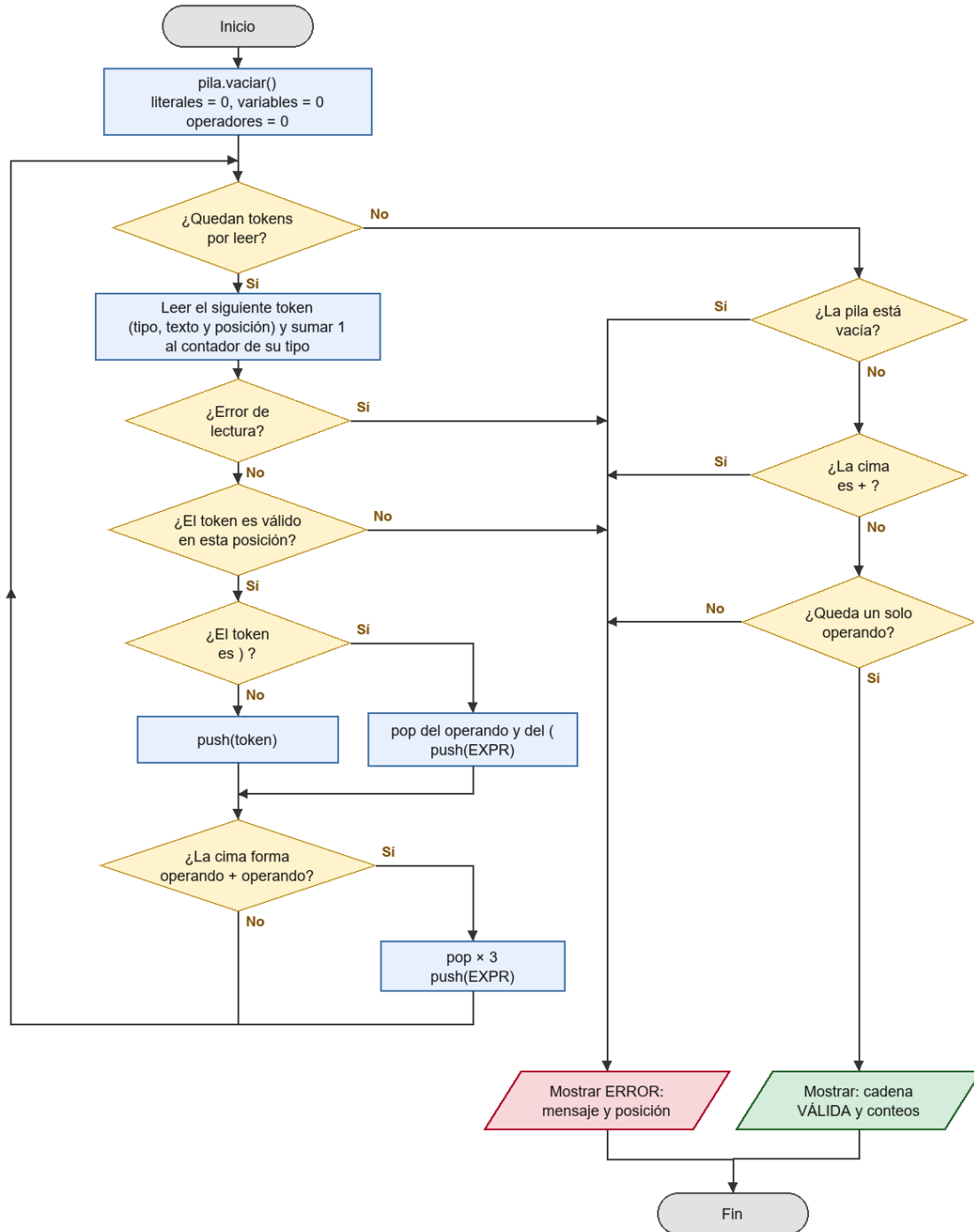


Figura 6. Flujo del método analizar(): lectura token por token, operaciones sobre la pila según el tipo de token y verificación final del estado de la pila.

6. Clases del programa

El proyecto está formado por cinco archivos de código. El código fuente completo se encuentra en el repositorio indicado en la portada y en la sección 8.

- **TipoToken** enumera los tipos de token: cadena literal, variable de cadena, operador de concatenación, paréntesis de apertura y de cierre, y EXPR, que representa una parte de la expresión ya reconocida.
- **Nodo** representa un elemento de la pila. Tal como indica la consigna para la alternativa seleccionada, cada nodo es un token de la cadena por analizar: guarda su tipo, su texto (por ejemplo "Hola" o nombre) y la posición donde empieza dentro de la expresión, dato que permite informar el lugar exacto de un error. También guarda una referencia al nodo situado debajo.
- **Pila** implementa la estructura con nodos enlazados. Ofrece push, pop, peek, estaVacía, getTamano, vaciar, contenido y mostrar. Las dos últimas permiten visualizar el estado de la estructura.
- **AnalizadorCadenas** contiene la lógica del análisis. Su método analizar lee la expresión token por token, inserta cada token en la pila, realiza las reducciones y verifica el estado final. Cuenta literales, variables y operadores, y puede emitir la traza paso a paso.
- **Main** es el entorno de ejecución. Su método main() invoca a menu(), que ofrece cinco opciones: analizar una cadena, analizarla mostrando la traza de la pila, ejecutar un conjunto de ejemplos, operar la pila manualmente mediante un submenú en el que el usuario escribe tokens para aplicar push, pop y peek, y salir. Las entradas no numéricas o fuera de rango se rechazan con un mensaje y no interrumpen el programa.

7. Conclusiones y recomendaciones

El análisis de cadenas de impresión muestra que la elección de la estructura de datos determina si el algoritmo puede ser correcto. El anidamiento de la expresión exige combinar primero los tokens más recientes, y la pila realiza esa política con costo constante por operación. La implementación resultante recorre la entrada una sola vez, usa memoria proporcional a la profundidad de anidamiento y produce diagnósticos precisos, con la misma técnica de desplazamiento y reducción que emplean los compiladores reales.

Para una extensión futura del trabajo se recomienda admitir secuencias de escape dentro de las cadenas literales, de modo que una comilla precedida por una barra invertida no se interprete como cierre. También sería útil reportar todos los errores de una expresión en lugar de detenerse en el primero, mediante una recuperación parecida a la que emplean los compiladores.

8. Repositorio del proyecto

El código fuente completo y ejecutable del proyecto de Java está disponible en el siguiente repositorio público de GitHub:

<https://github.com/B0W3YYY/CasoDeEstudio1-EstructuraDeDatos>

El repositorio contiene la carpeta `src` con los archivos `TipoToken.java`, `Nodo.java`, `Pila.java`, `AnalizadorCadenas.java` y `Main.java`, junto con un archivo `README.md` que explica cómo compilar y ejecutar el programa.

9. Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2.^a ed.). Pearson/Addison-Wesley.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.^a ed.). The MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6.^a ed.). John Wiley & Sons.

Oracle. (2023). *The Java Language Specification, Java SE 21 Edition*. Sección 3.10.5, String Literals.
<https://docs.oracle.com/javase/specs/jls/se21/html/jls-3.html>

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.^a ed.). Addison-Wesley.